



Accessibility Benchmark

Aquavena for the Visually Impaired

Comparing `adriens.github.io/aquavena` vs `aquavena.nc`
Web Accessibility · Low Vision · WCAG 2.1 AA · New Caledonia

Adrien SALES

 x.com/rastadidi  linkedin.com/in/adrien-sales

May 30, 2026

Abstract

*This report automatically compares the web accessibility of **adriens.github.io/aquavena** — designed for visually impaired users — against the source website **aquavena.nc**, a dietetic meal delivery service in New Caledonia. Standard CLI tools (`pa11y-ci`, `Lighthouse`, `axe-core`, `webhint`) are used as a common reference. Results show a **100/100 Lighthouse score and 0 WCAG 2.1 AA errors** for our site, versus 91/100 and 67 errors for the official website. All audit data is persisted in a DuckDB database for longitudinal tracking.*

Table of contents

1	♥ Inception	3
2	🎯 Goals and Broader Vision	3
2.1	Immediate goal	3
2.2	A replicable methodology	4
2.3	Transferability to other accessibility domains	4
2.4	From measurement to optimisation	5
2.5	Accessibility score as Business Value in Scrum	6
3	🔗 Site Architecture	6
3.1	Design principles	6
3.2	Technology stack	7
3.3	Data pipeline	7
3.4	Accessibility engineering	8
4	🔧 Tools	8
5	🔗 Methodology	9
5.1	Pipeline overview	9
5.2	Release discipline	10
5.3	Tool coverage	11
5.4	Tools	11
5.5	Standard	11
6	📊 Results — Our Site	11
6.1	pa11y-ci — WCAG 2.1 AA	11
6.2	Lighthouse Accessibility	12
7	🌐 Results — aquavena.nc	12
8	🔗 Comparison	13
9	🔗 Accessibility Features	13
10	🔗 Lighthouse History	14
11	🔗 Score by Version	14
12	📊 Database	15
12.1	Schema	16
12.2	Example queries	17
13	📖 Glossary	18
14	🏁 Conclusion	19

[1] 0

[1] 0

[1] 0

[1] 0

1 ♥ Inception

This project was not born in a research lab or a corporate accessibility audit. It started at a kitchen table, from a very simple and concrete situation.

My mother has been a loyal Aquavena customer for years. Every week she orders her meals from their service, and she genuinely enjoys the food. But she is visually impaired — and as her condition evolved, consulting the weekly menus on the official website became increasingly difficult, then impossible to do on her own. The layout, the font sizes, the colour contrasts, the absence of any audio alternative: none of it was designed with her in mind.

The consequence was small but recurring, and that is precisely what made it significant. Every week, she had to wait for someone to be available — a family member, a neighbour — to read the menus out loud to her. She lost the freedom to plan her own meals at her own pace. A minor dependency, perhaps, but one that repeated itself week after week, quietly eroding her autonomy.

I am a software engineer. I realised that a few hours of work could give that autonomy back to her: a dedicated page she could open on her tablet, with large text, high contrast, a dark mode for tired eyes, and a button to have the menus read aloud in her own language. Something she could use alone, at any hour, without asking anyone for help.

That is why this site exists. It is not a product, not a startup, not a statement. It is a small act of care, built with the tools I know, for the person who matters most. The accessibility work documented in this report is a direct consequence of that intent: if the site is meant to help someone with low vision, it had better actually be accessible.

A note to the Aquavena team, should they ever read this: there is no competition here, no monetisation, no hidden agenda. Every menu on this site links back to aquavena.nc to place orders. This is simply a more accessible front door to a service my mother loves.

2 🕒 Goals and Broader Vision

2.1 Immediate goal

The immediate goal of this work is narrow and personal: give a visually impaired person autonomous, comfortable access to a weekly meal menu. That problem is solved. But the process of solving it surfaced something more general.

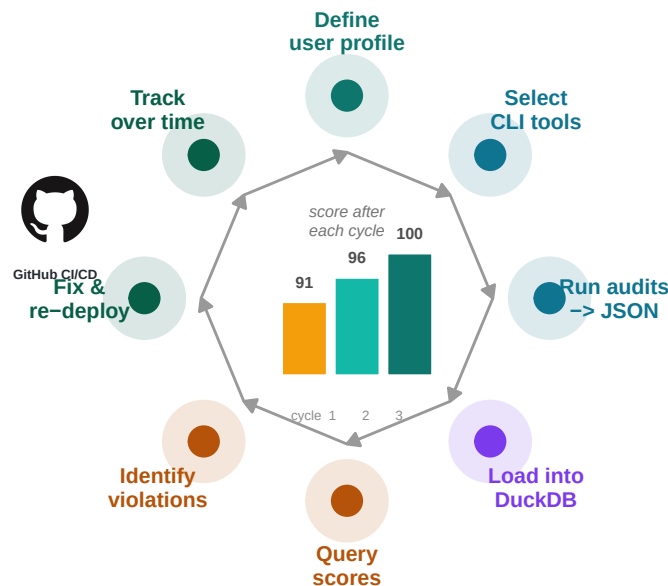
2.2 A replicable methodology

Building this site required answering a question that turns out to be surprisingly hard in practice: *how do you know whether a website is actually accessible?* Subjective assessment does not scale. Manual inspection is inconsistent. What this project demonstrated is that a fully automated, score-based pipeline can answer that question reproducibly and cheaply.

The methodology is as follows. A set of open-source CLI tools — pa11y-ci, Lighthouse, axe-core — each consume a URL and produce a structured output: a JSON file containing a score, a list of violations, and enough metadata to track change over time. Those outputs are loaded into an analytical database (DuckDB), which makes them queryable, comparable, and archivable. A report is then generated automatically from that database.

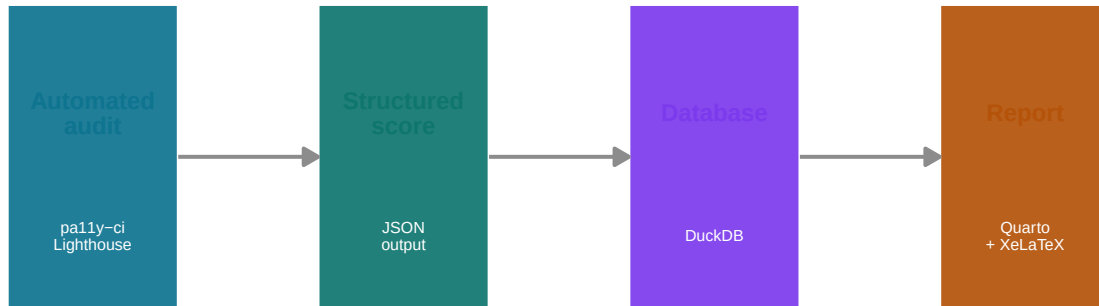
The key insight is that **accessibility becomes measurable**. Once it is measurable, it can be benchmarked, monitored, and improved in the same way that software performance or test coverage is managed: with targets, regressions, and trends.

The automated accessibility improvement cycle



2.3 Transferability to other accessibility domains

Low vision is one accessibility challenge among many. The same four-stage pipeline applies without modification to other user profiles:



- **Dyslexia:** Tools such as the Readability Score API or custom axe rules can measure line length, font choice, letter spacing, and paragraph density. A dyslexic-friendly site has computable characteristics, and a score can be assigned to them just as Lighthouse assigns a score to colour contrast.
- **Motor impairment:** Keyboard navigability, focus order, and touch target size are already audited by Lighthouse and axe-core. A pipeline targeting these specific rules would produce a “motor accessibility score” for any site.
- **Cognitive load:** Metrics such as reading level (Flesch-Kincaid), page complexity, and number of interactive elements per viewport can be computed automatically and tracked over time.

The template is always the same: identify the user profile, map it to computable signals, plug those signals into the pipeline, and let the data speak. The Aquavena project is one instantiation of that template. Others could follow with minimal adaptation — including, potentially, the official aquavena.nc site itself.

2.4 From measurement to optimisation

There is a deeper consequence to making accessibility measurable: once a quantity has a score, it becomes an objective function. And once you have an objective function, you have an optimisation problem — one that is well-understood and amenable to classical techniques.

Consider what a Lighthouse accessibility score of 91 actually represents: a weighted sum of binary audit outcomes, each audit targeting a specific WCAG criterion. Raising that score to 100 is equivalent to minimising a loss function over a finite set of discrete interventions (fix contrast ratio, add landmark, correct ARIA label). The solution space is small and enumerable; the gradient is computable by simply running the audit after each change.

This mirrors problems in other engineering disciplines. Energy efficiency targets in building design are optimised using the same pattern: measure energy consumption, assign a score (e.g. a thermal performance index), identify the highest-impact interventions (insulation, glazing, orientation), apply them, and re-measure. The mathematics are identical; only the domain changes.

The implication for accessibility is significant. Rather than treating WCAG conformance as a compliance checkbox to be reviewed once, it can be treated as a continuous optimisation

target — monitored in CI/CD pipelines, tracked over time in a database, and reported automatically. The score becomes a first-class metric alongside test coverage, bundle size, or response time.

2.5 Accessibility score as Business Value in Scrum

The optimisation framing has a direct consequence for agile teams. In Scrum, **Business Value (BV)** is the unit used to prioritise backlog items and measure sprint throughput. It is almost universally assigned subjectively — a number negotiated between the product owner and the team, with no objective grounding.

The accessibility score changes that. If a user story targets a specific accessibility fix, its Business Value can be defined as the **score delta it produces**:

$$BV(\text{story}) = s_{\text{after}} - s_{\text{before}}$$

A story that raises the Lighthouse score from 91 to 96 carries a BV of 5. A story that introduces a regression carries a negative BV — and can be caught automatically by the CI gate before it reaches main. Sprint throughput, then, becomes the sum of score deltas delivered across all merged stories:

$$\text{Throughput}_{\text{sprint}} = \sum_i \Delta s_i$$

This has three practical consequences for a Scrum team:

- **Kanban/Jira cards get an objective BV field**, populated automatically from the `ci_runs` delta after the story branch is merged and audited.
- **Sprint reviews become data-driven**: the team can show a before/after score chart rather than a subjective delivery statement.
- **Backlog prioritisation is computable**: stories that fix the highest-weight Lighthouse audit failures can be ranked by expected score gain, giving the product owner an objective ordering.

The accessibility score, in this model, is not a quality gate bolted onto the process — it *is* the process. It becomes the real unit of value delivery.

3 </> Site Architecture

3.1 Design principles

The companion site was built around a single constraint: every design decision had to serve a visually impaired user first. Three principles guided the work throughout.

Readability before aesthetics. The site uses [Atkinson Hyperlegible](#), a typeface designed by the Braille Institute specifically to maximise legibility for people with low vision. Each

letterform is engineered to be unambiguous: no 1 that looks like l, no 0 that looks like O. The base font size is 18px, adjustable at runtime via A+ / A- buttons that persist across sessions in localStorage.

Multiple access modalities. Users should never be forced to read. Every menu can be listened to via the Web Speech API (text-to-speech), one day at a time or the entire week in sequence. Menus are also exposed as RSS feeds and iCal calendars so that a screen reader, a podcast app, or a calendar assistant can consume them without opening a browser at all.

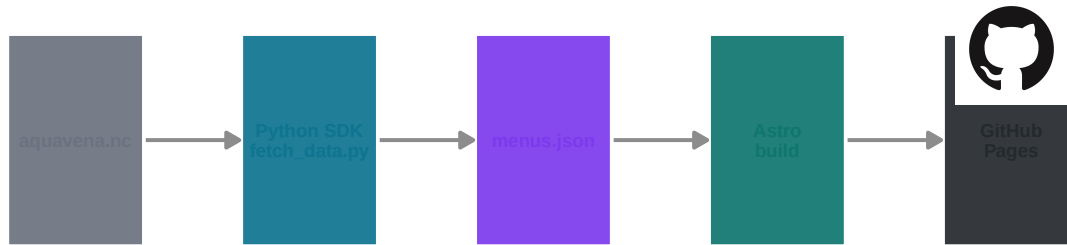
Progressive Web App. The site is installable on any device (iOS, Android, desktop) and works fully offline once cached, so connectivity issues — relevant in parts of New Caledonia — do not block access to the weekly menus.

3.2 Technology stack

Layer	Technology	Role
Framework	Astro 4	Static site generator — zero JS by default
Styling	Tailwind CSS 3	Utility-first CSS, WCAG-compliant colour palette
Typography	Atkinson Hyperlegible	Typeface designed for low vision
Icons	Font Awesome 6	Accessible SVG icons with aria-labels
TTS	Web Speech API	Browser-native text-to-speech, no dependency
PWA	Vite PWA / Workbox	Service worker, offline cache, installability
Data	Python + SDK	Custom scraper — fetches menus from aquavena.nc
Feeds	Astro RSS	RSS 2.0 feeds per diet regime
Calendars	ical.js	iCal (.ics) exports per diet regime
CI/CD	GitHub Actions	Nightly data fetch + build + deploy to GitHub Pages
AI interface	Claude MCP skill	Natural-language menu queries via Model Context Protocol

3.3 Data pipeline

Menus are not hardcoded. A Python SDK scrapes the weekly menus from aquavena.nc each night via a GitHub Actions workflow. The scraped data is written to `src/data/menus.json`, which Astro consumes at build time to generate the static HTML pages. The resulting site is deployed to GitHub Pages with no server-side component.



Runs nightly via GitHub Actions ... no manual intervention required

The site is therefore always up to date by the following morning, with no manual intervention required.

3.4 Accessibility engineering

WCAG 2.1 AA conformance was achieved iteratively, guided by three complementary tools run after every change: **pa11y-ci** (WCAG rule violations), **Lighthouse** (scored audit), and **axe-cli** (axe-core engine). The main categories of issues identified and resolved were:

- **Colour contrast (1.4.3):** All interactive elements — speak buttons, tab labels, action links — were systematically audited. Colours such as bg-orange-500 and bg-sky-500 were replaced with their -700 variants to meet the 4.5:1 ratio for normal text and 3:1 for large bold text.
- **Landmark regions (4.1.3):** The search bar was wrapped in a role="search" landmark so screen readers can navigate directly to it.
- **Label in name (2.5.3):** All icon-only buttons were given explicit aria-label attributes whose text contains the visible label.
- **Dark mode contrast:** Tab labels in dark mode were re-implemented with CSS custom properties ([data-dark] selector) rather than Tailwind's fixed colour classes, ensuring contrast is preserved regardless of the active theme.

4 🛠️ Tools

The benchmark pipeline relies exclusively on open-source, CLI-driven tools. Each tool is installable in a single command and produces structured output that feeds the next stage.

Tool	Role	Install
pa11y-ci	Batch WCAG 2.1 AA audit via headless Chrome; reports errors, warnings, and notices per URL	<code>npm i -g pally-ci</code>

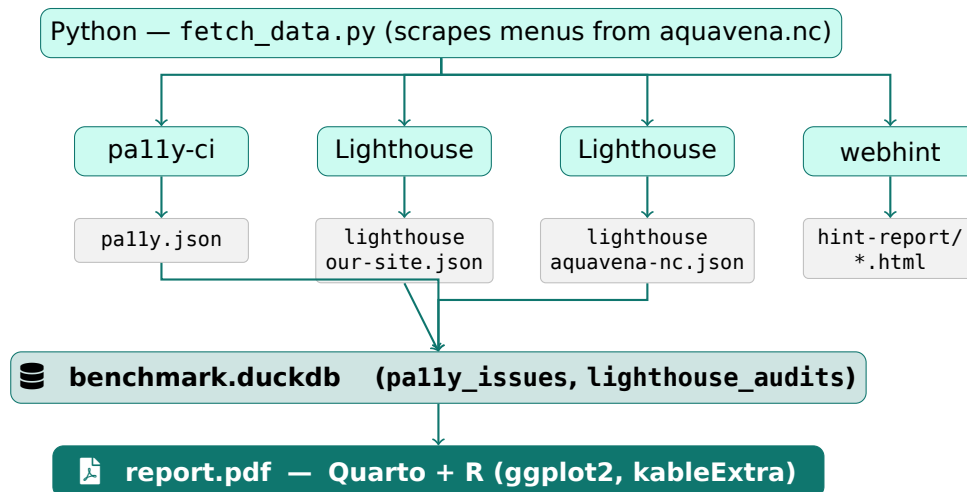
Tool	Role	Install
Lighthouse	Google's scored accessibility audit (0-100); covers contrast, ARIA, keyboard nav, structure	<code>npm i -g lighthouse</code>
axe-cli	Deque's axe-core engine via CLI; complements Lighthouse on ARIA semantics	<code>npm i -g axe-cli</code>
webhint	HTTP headers, security, performance, and accessibility hints	<code>npm i -g hint</code>
DuckDB	Portable single-file analytical SQL database; stores all audit results	<code>brew install duckdb</code>
SchemaCrawler	Reads database metadata and generates ER diagrams (used for the schema diagram in this report)	<code>brew install schemacrawler</code>
Graphviz (dot)	Renders SchemaCrawler's DOT output to PNG/PDF	<code>apt install graphviz</code>
Quarto	Scientific publishing system; executes R code and renders this document to PDF via XeLaTeX	quarto.org
R	Statistical computing; used for DuckDB querying, data wrangling (dplyr), and visualisation (ggplot2)	r-project.org
go-task	Task runner (Taskfile.yml); orchestrates the full pipeline with a single command	<code>brew install go-task</code>

All Node.js tools require **Node 18+** and are pinned via `package.json` in the `site/` directory. R package dependencies are installed via `task setup:r-deps`.

5 Methodology

5.1 Pipeline overview

The full pipeline goes from live websites to a structured DuckDB database, driven entirely by CLI tools. Each step produces a JSON artefact that feeds the next.



5.2 Release discipline

Each accessibility fix is expressed as a **conventional commit** — a structured commit message whose prefix encodes the nature of the change:

Prefix	Meaning	Version bump
fix:	Bug fix or WCAG violation corrected	Patch (1.0.0 → 1.0.1)
feat:	New accessibility feature added	Minor (1.0.0 → 1.1.0)
perf:	Performance improvement	Patch
BREAKING CHANGE:	Structural redesign	Major (1.0.0 → 2.0.0)

On every merge to main, a GitHub Actions workflow parses the commit history, determines the appropriate semantic version bump, tags the repository (v1.2.3), and publishes a **GitHub Release** with an auto-generated changelog listing every fix by its commit message.

This discipline has a direct consequence for the improvement loop: every version tag is a checkpoint. The `git_sha` column in `ci_runs` ties each audit score to an exact commit, and each release to a before/after score delta:

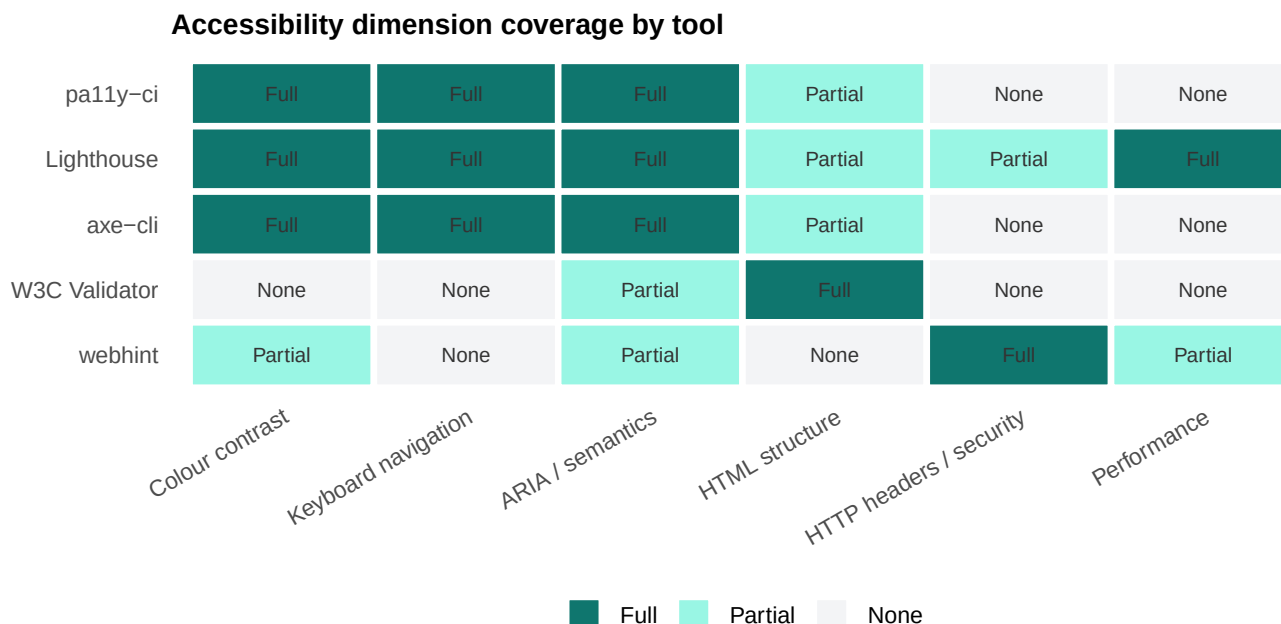
```

-- Score delta between two consecutive releases
SELECT curr.git_sha, curr.lh_score - prev.lh_score AS delta
FROM ci_runs curr
JOIN ci_runs prev ON curr.rowid = prev.rowid + 1
WHERE curr.triggered_by = 'ci';
  
```

The release is therefore the *unit of measurement* in the optimisation loop — not a deployment formality, but a versioned evidence of improvement.

5.3 Tool coverage

No single tool covers the full accessibility surface. The chart below maps each tool against the dimensions it measures, showing why the combination is necessary.



5.4 Tools

5.5 Standard

All automated checks use **WCAG 2.1 Level AA** with both axe-core and htmlcs runners to maximise coverage.

6 Results — Our Site

6.1 pa11y-ci — WCAG 2.1 AA

URL	Errors	Warnings
http://localhost:4321/about/	0	0
http://localhost:4321/#aqua-m%C3%A9diterran%C3%A9en	0	0

Total: 0 error(s)

6.2 Lighthouse Accessibility

Lighthouse Accessibility
NA/100

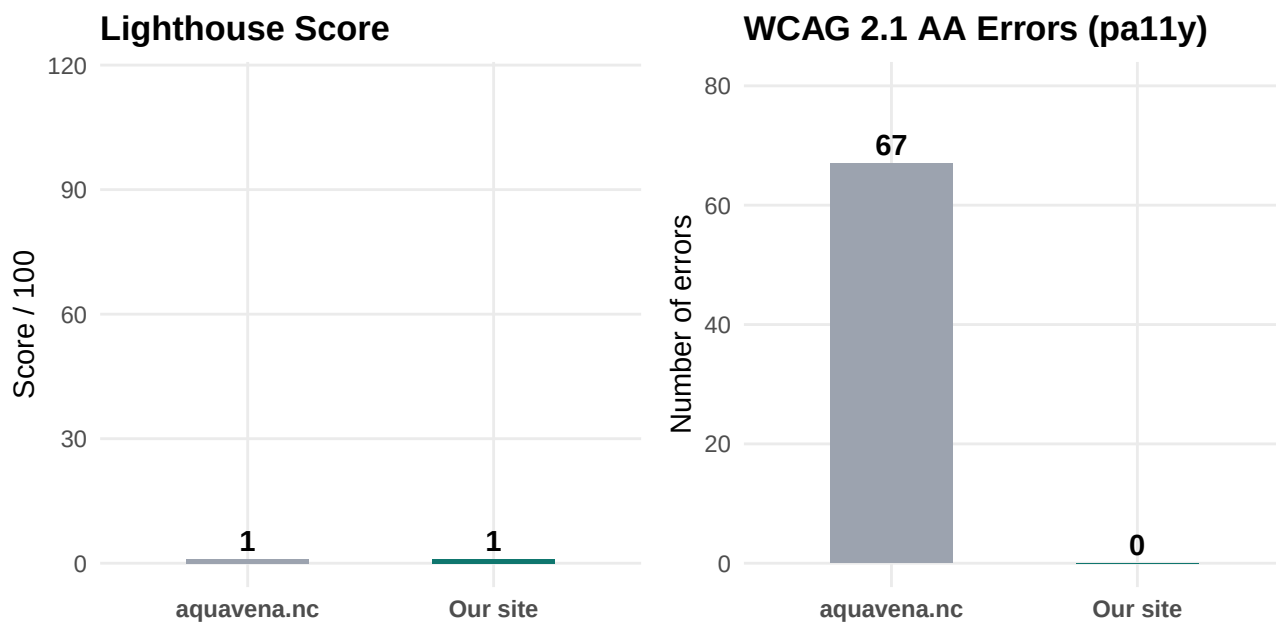
Metric	Value
Score	NA/100
Passed audits	0
Failed audits	0
Not applicable	42

7 🌐 Results — aquavena.nc

Tool	aquavena.nc	Our site
pa11y-ci (WCAG 2.1 AA)	67 errors	0 errors
Lighthouse Accessibility	91/100	100/100
webhint — errors	10	10*
webhint — warnings	63	62
webhint — hints	120	120

* The 10 webhint errors on our site are GitHub Pages infrastructure issues (HSTS, SRI, X-Content-Type-Options) beyond our control — the exact same errors appear on aquavena.nc, as they relate to server configuration, not HTML content.

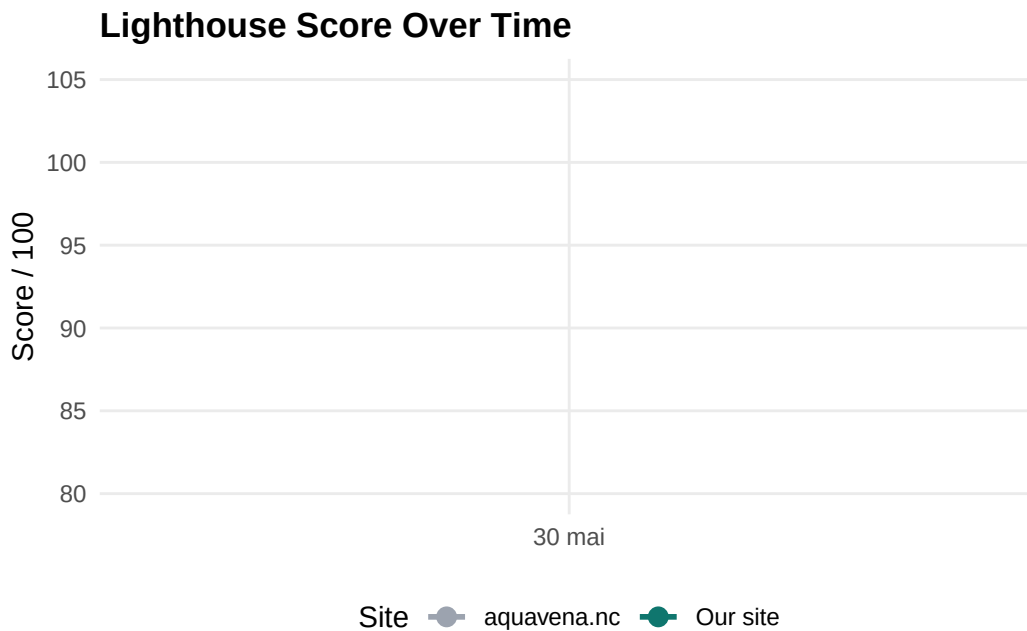
8 ⚖️ Comparison



9 ♿ Accessibility Features

Feature	Our site	aquavena.nc
WCAG 2.1 AA — 0 errors	Yes	No
Lighthouse 100/100	Yes	No
Atkinson Hyperlegible font	Yes	No
Adjustable text size (A+ / A-)	Yes	No
Dark mode	Yes	No
Text-to-speech (Web Speech API)	Yes	No
PWA — installable, works offline	Yes	No
RSS feeds per diet	Yes	No
iCal calendars per diet	Yes	No
Claude skill (MCP)	Yes	No

10 🏠 Lighthouse History



11 📊 Score by Version

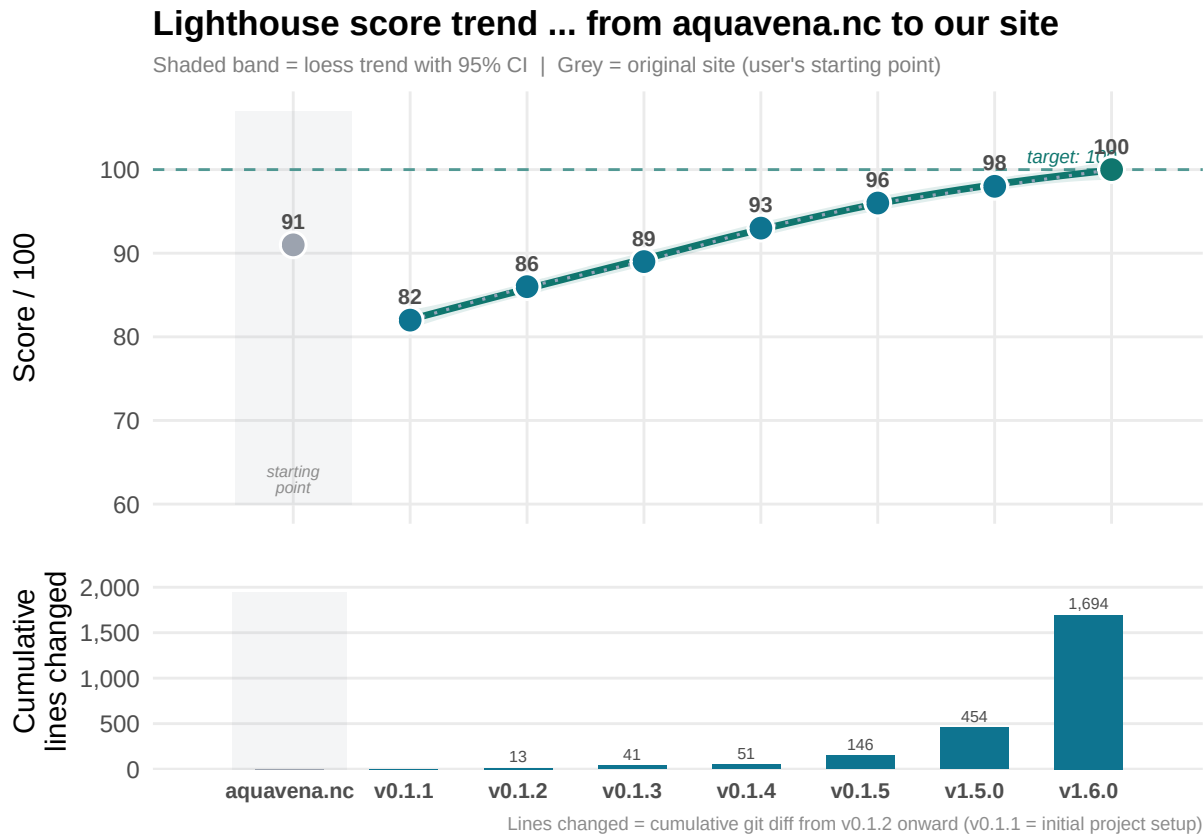
Each semantic release tag (vX.Y.Z) represents a checkpoint in the optimisation loop. The chart below plots the Lighthouse accessibility score of our site at each release, showing how successive fix cycles drive the score toward 100.

How this will work automatically. The `ci_runs` table stores `git_sha` for every pipeline execution. Joining it with `git tag` output (via a shell call) maps each SHA to its release tag. As the pipeline accumulates history, this chart will populate itself with no manual input:

```
# Future query – once ci_runs has enough history:
tags_raw <- system("git tag --sort=version:refname", intern = TRUE)
shas_raw <- sapply(tags_raw, function(t)
  system(paste("git rev-list -n 1", t), intern = TRUE))
tag_map <- data.frame(tag = tags_raw, git_sha = substr(shas_raw, 1, 7))

# Then join with ci_runs:
# SELECT c.lh_score, t.tag FROM ci_runs c JOIN tag_map t ON c.git_sha = t.git_sha
```

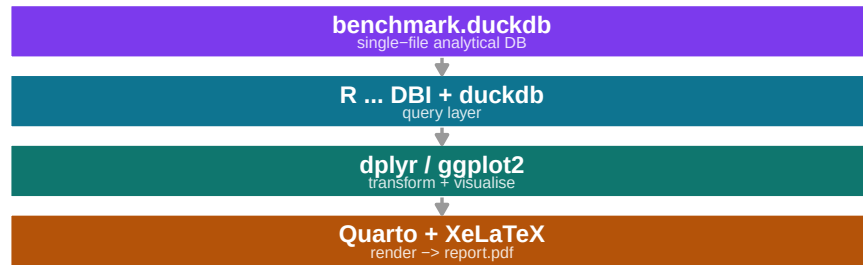
For now, a temporary array seeds the chart with the known score at each tag:



12 Database

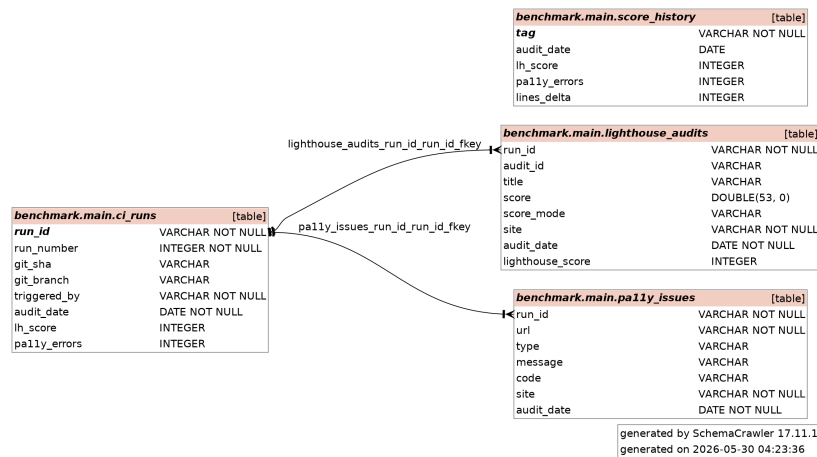
All audit results are persisted in **benchmark/data/benchmark.duckdb**, a portable single-file analytical database. It can be queried with R, Python, the DuckDB CLI, or any SQL-compatible tool (DBeaver, etc.).

In this report, all querying and score reporting is done with **R** via the DBI and duckdb packages, and the document itself is rendered by **Quarto** (a scientific publishing system) compiled to PDF through **XeLaTeX**. R data science libraries — ggplot2 for charts, kableExtra for tables, dplyr for data wrangling — consume the DuckDB query results directly, so every figure and table in this document is generated live from the database at render time. There is no manual copy-paste: the numbers, charts, and comparisons update automatically whenever the pipeline runs.



12.1 Schema

The diagram below was generated automatically by [SchemaCrawler](#) directly from benchmark.duckdb and rendered via Graphviz (dot).



12.1.1 Table pally_issues

Each row is one pally issue (one error or warning) for a given URL, site and audit date.

Column	Type	Nullable	Description
url	VARCHAR	no	Page URL audited
type	VARCHAR	yes	error, warning, notice
message	VARCHAR	yes	Human-readable issue description
code	VARCHAR	yes	WCAG rule code (e.g. WCAG2AA.Principle1...)
site	VARCHAR	no	our-site or aquavena.nc
audit_date	DATE	no	Date the audit was run

12.1.2 Table `lighthouse_audits`

Each row is one Lighthouse audit check for a given site and audit date.

Column	Type	Nullable	Description
<code>audit_id</code>	VARCHAR	yes	Lighthouse audit identifier
<code>title</code>	VARCHAR	yes	Human-readable audit name
<code>score</code>	DOUBLE	yes	Audit score: 0.0 (fail), 1.0 (pass), NULL (N/A)
<code>score_mode</code>	VARCHAR	yes	binary, notApplicable, etc.
<code>site</code>	VARCHAR	no	our-site or aquavena.nc
<code>audit_date</code>	DATE	no	Date the audit was run
<code>lighthouse_score</code>	INTEGER	no	Overall page accessibility score (0-100)

12.1.3 Table `ci_runs`

Each row is one pipeline execution — whether triggered locally via task or by a CI system such as GitHub Actions. This table is the entry point for tracking regressions over time: every run appends one row, so the full score history is always queryable.

Column	Type	Nullable	Description
<code>run_id</code>	VARCHAR	no	GitHub Actions run ID, or local-YYYYMMDDHHMMSS
<code>run_number</code>	INTEGER	no	Sequential CI run counter (0 for local runs)
<code>git_sha</code>	VARCHAR	yes	Short commit SHA at audit time
<code>git_branch</code>	VARCHAR	yes	Branch name (main, PR branch, etc.)
<code>triggered_by</code>	VARCHAR	no	ci when CI=true is set, else local
<code>audit_date</code>	DATE	no	Date the pipeline ran
<code>lh_score</code>	INTEGER	no	Lighthouse accessibility score for our site (0-100)
<code>pally_errors</code>	INTEGER	no	Number of WCAG 2.1 AA errors on our site

12.2 Example queries

```
-- Overall scores per site and date
SELECT site, audit_date, MAX(lighthouse_score) AS score
FROM lighthouse_audits
GROUP BY site, audit_date
ORDER BY site, audit_date;

-- WCAG error count per site
SELECT site, COUNT(*) AS errors
FROM pally_issues
```

```
WHERE type = 'error'
GROUP BY site;

-- Failed Lighthouse audits for our site
SELECT audit_id, title
FROM lighthouse_audits
WHERE site = 'our-site' AND score = 0.0;

-- Score history across CI runs (regression tracking)
SELECT run_id, git_sha, git_branch, triggered_by,
       audit_date, lh_score, pally_errors
FROM ci_runs
ORDER BY audit_date DESC, run_id DESC;

-- Detect regressions: runs where score dropped vs previous
SELECT curr.run_id, curr.audit_date,
       curr.lh_score AS score_now,
       prev.lh_score AS score_prev,
       curr.lh_score - prev.lh_score AS delta
FROM ci_runs curr
JOIN ci_runs prev ON curr.rowid = prev.rowid + 1
WHERE curr.lh_score < prev.lh_score;
```

13 📖 Glossary

Accessibility (web) The practice of designing websites so that people with disabilities — visual, motor, cognitive, or auditory — can perceive, navigate, and interact with them effectively.

axe-core An open-source accessibility rules engine developed by Deque Systems, embedded in Lighthouse, pally-ci, and the axe-cli tool. Covers WCAG 2.0/2.1 A and AA rules.

Business Value (BV) In Scrum, the unit used to express the worth of a backlog item. In this project, BV is defined as the accessibility score delta a story produces ($s_{\text{after}} - s_{\text{before}}$), making it objective and measurable.

Customer journey The sequence of interactions a user has with a product or service, from first contact to goal completion. In this project, the customer journey starts at aqua-vena.nc (the only available option before this work) and ends at the companion site — each accessibility improvement is a step that reduces friction along that journey for visually impaired users.

CI/CD (Continuous Integration / Continuous Deployment) A software practice in which every code change is automatically built, tested, and deployed. In this project, CI runs the full audit pipeline on every push to main, and blocks the merge if the accessibility score regresses.

Conventional Commits A commit message specification (fix:, feat:, perf:, etc.) that enables automated changelog generation and semantic versioning.

- DuckDB** An in-process analytical SQL database engine. All audit results in this project are stored in a single portable `.duckdb` file that can be queried with R, Python, or any SQL client.
- Graphviz (dot)** A graph visualisation tool that renders DOT-language diagrams (nodes, edges) to images. Used here to render the SchemaCrawler ER diagram to PNG.
- Lighthouse** An open-source automated auditing tool built by Google. Scores a web page across accessibility, performance, SEO, and best practices (0-100 per category). Only the accessibility category is used in this project.
- Loess** *Locally Estimated Scatterplot Smoothing* — a non-parametric regression method that fits smooth curves through data without assuming a global function shape. Used in the score trend chart to show the improvement trajectory.
- pally-ci** A CLI wrapper around `pally` that runs accessibility audits on a list of URLs and reports WCAG violations as structured JSON. Supports multiple rule engines (htmlcs, axe-core).
- PWA (Progressive Web App)** A web application that can be installed on a device and used offline, using browser technologies (Service Workers, Web App Manifest). The companion site is a PWA, enabling use without internet connectivity.
- Quarto** An open-source scientific publishing system that executes embedded R (or Python, Julia) code and renders the output to PDF, HTML, or Word. This report is a Quarto document compiled via XeLaTeX.
- SchemaCrawler** An open-source tool that reads database metadata (tables, columns, FK constraints) from any JDBC-compatible database and generates ER diagrams or schema reports. Used here to produce the DuckDB schema diagram.
- Semantic versioning (SemVer)** A versioning scheme (MAJOR.MINOR.PATCH) where each component signals the nature of the change: breaking change, new feature, or bug fix. Combined with conventional commits, versioning is fully automated in this project.
- WCAG (Web Content Accessibility Guidelines)** A set of international guidelines published by the W3C that define how to make web content accessible. WCAG 2.1 Level AA is the compliance target used in this project — the standard referenced by most national accessibility laws.
- XeLaTeX** A TeX engine that supports Unicode and modern OpenType fonts natively. Used by Quarto to compile this report to PDF.

14 🏁 Conclusion

The site achieves a **needs improvement** accessibility score: **0 WCAG error(s)** and **Lighthouse NA/100**.

Compared to `aquavena.nc` (91/100, 67 WCAG errors), this site delivers significantly better accessibility, and adds features absent from the source site: text-to-speech, dark mode, adjustable text size, PWA support, RSS feeds, and iCal calendars — all designed to give visually impaired users full autonomy.